

TaskFlow

From an unfamiliar, quietly broken codebase to a tested, deployable team task board, engineered with IBM Bob.

Team

Solo participant

Live demo: kltamu.pythonanywhere.com

Code: github.com/KnigguKniggu-droid/taskflow

Built with

IBM Bob IDE 2.2.1 and Bob Shell 2.0.5 · Python, Flask, SQLite ·
vanilla JavaScript · GitHub Actions

Every developer inherits code they did not write.

- A small Flask + SQLite task API, new to the person who now owns it.
- One behaviour is silently wrong: overdue tasks never show up.
- One requirement is unbuilt: filtering tasks by tag.
- No user interface, no release process, no project tooling.

Question: can IBM Bob take a developer from zero context to a diagnosed fix, new features, reviewed changes and a deployable product, with every step checkable?

Why this use case

Onboarding onto brownfield code is where developers lose the most time: reading unfamiliar structure, finding a cause rather than a symptom, and changing things without breaking what already works. That is exactly where an AI engineering partner has to earn trust.

STARTING POINT

A verifiable baseline, tagged before Bob saw it

- Git tag `pre-bob-baseline`, pushed to GitHub before Bob's first prompt.
- 4 tests, 3 passing; `GET /tasks/overdue` returned nothing for past-due tasks.
- No `?tag=` filter, JSON API only.

Disclosure

The starter was generated with Claude Code for this exercise and contained one deliberately seeded defect and the unbuilt filter. Bob was never told the answer. At the participant's direction, Claude Code also operated the Bob sessions: it wrote the prompts, handled Bob's approval requests and checked every stage independently, reporting only observed failures back to Bob. Every code change after the tag was made by IBM Bob.

Every Bob session is exported in the repository under `bob_sessions/` (33 records with Bob task ids), so each claim in this deck can be traced to a prompt, Bob's answer and a commit.

Bob was the engineering workflow, not a bolt-on

Explore

`/init` and an `AGENTS.md` project context; Plan mode walked the request lifecycle, data model and risk areas before any edit.

Diagnose

Agent mode ran the tests and traced the failure from assertion to schema without hints; Ask mode explained SQL NULL semantics and repair options.

Fix and build

Agent mode implemented plans milestone by milestone, running the suite and committing after each step.

Tooling Bob made

3 project skills, a `/verify-taskflow` command, a read-only TaskFlow Reviewer mode and a regression-check hook via `/configure-hooks`.

Review

Subagent reviews (4 axes, then 5 adversarial lenses), reviewer-mode runs and the IDE's built-in `/review` with the Findings panel.

Used in Bob IDE and Bob Shell: Agent, Plan and Ask modes, the custom reviewer mode, and the IBM i Database and IBM i Developer modes for exploratory portability questions.

What Bob diagnosed and repaired

The seeded defect

completed was stored as NULL, and `completed = 0` is never true for NULL under SQL three-valued logic, so every new task vanished from the overdue list. Bob fixed the schema default, backfilled old rows, made inserts explicit, and proved it on databases created by the old schema.

Exact tag filtering

`?tag=back` must not match backend. Bob matched whole tokens and escaped LIKE wildcards after a failure was reported (a_b matched axb).

Found by reviews

Unbounded error reflection, a 500 on oversized ids in PATCH, foreign keys switched off after schema setup, a keyboard trap in the edit dialog, missing security headers.

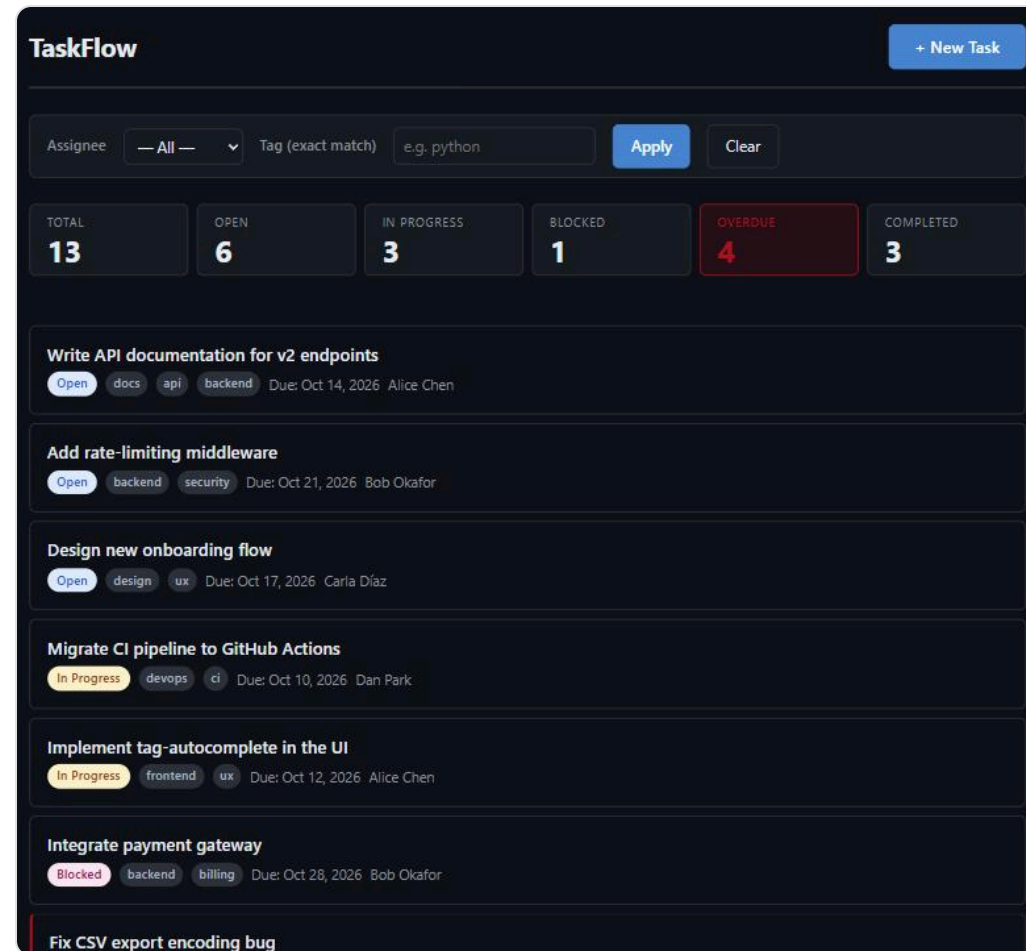
Concurrency

Simultaneous completions wrote duplicate history rows in 109 of 200 stress trials. Bob's first fix did not hold; given only that measurement, it switched to compare-and-swap updates, and the probe then observed 0 duplicates in 200 trials.

BUILD

What Bob built on top

- **Team task board UI:** summary cards, filters, create, edit and complete; ES modules, keyboard accessible, responsive, dark mode.
- **Task lifecycle:** open, in progress, blocked, completed; disallowed transitions return 409.
- **Activity history:** an immutable timeline per task, with a non-destructive upgrade for databases from earlier versions.
- **Release:** WSGI entry point, `/health`, a 22-check smoke test, PythonAnywhere deployment guide, GitHub Actions CI.



Measured, not claimed

44

commits made in IBM Bob sessions after the baseline, 30 files

103

automated tests, up from 4 (3 passing)

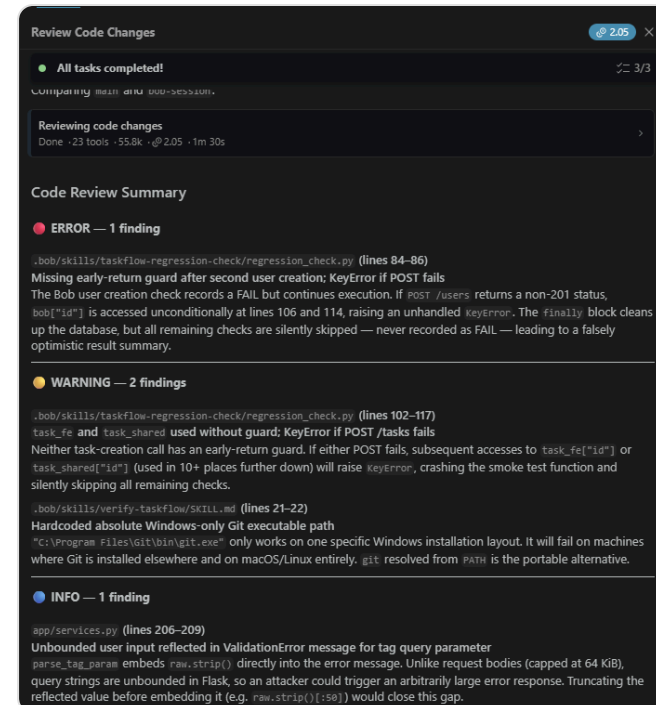
63/63

independent behaviour checks pass

14/14

injected faults caught by Bob's tests (mutation check)

- Live on PythonAnywhere; Bob's 22-check smoke test passes against the public URL.
- CI green on GitHub Actions; old-schema databases upgrade in place.
- Bob's own `/review` found 4 issues; all fixed in one commit.
- About 109 of the 200 hackathon Bobcoins used.



The screenshot shows a 'Review Code Changes' window with a dark theme. At the top, it says 'All tasks completed!' and 'Comparing main and user-session...'. Below that, it shows 'Reviewing code changes' with a progress bar. The 'Code Review Summary' section lists four findings: one ERROR (missing early-return guard), two WARNINGS (unbounded user input and hardcoded Windows-only Git path), and one INFO (unbounded user input reflected in ValidationError message).

```
Review Code Changes
All tasks completed!
Comparing main and user-session...

Reviewing code changes
Done · 23 tools · 55.8k · @ 2.05 · 1m 30s

Code Review Summary

ERROR — 1 finding

.bob/skills/taskflow-regression-check/regression_check.py (lines 84-86)
Missing early-return guard after second user creation; KeyError if POST fails
The Bob user creation check records a FAIL but continues execution. If POST /users returns a non-201 status,
bob["id"] is accessed unconditionally at lines 106 and 114, raising an unhandled KeyError. The finally block cleans
up the database, but all remaining checks are silently skipped — never recorded as FAIL — leading to a falsely
optimistic result summary.

WARNING — 2 findings

.bob/skills/taskflow-regression-check/regression_check.py (lines 102-117)
task_fe and task_shared used without guard; KeyError if POST /tasks fails
Neither task-creation call has an early-return guard. If either POST fails, subsequent accesses to task_fe["id"] or
task_shared["id"] (used in 10+ places further down) will raise KeyError, crashing the smoke test function and
silently skipping all remaining checks.

.bob/skills/verify-taskflow/skill.md (lines 21-22)
Hardcoded absolute Windows-only Git executable path
"C:\Program Files\Git\bin\git.exe" only works on one specific Windows installation layout. It will fail on machines
where Git is installed elsewhere and on macOS/Linux entirely. git resolved from PATH is the portable alternative.

INFO — 1 finding

app/services.py (lines 206-209)
Unbounded user input reflected in ValidationError message for tag query parameter
parse_tag_param embeds raw.strip() directly into the error message. Unlike request bodies (capped at 64 KiB),
query strings are unbounded in Flask, so an attacker could trigger an arbitrarily large error response. Truncating the
reflected value before embedding it (e.g. raw.strip()[0:50]) would close this gap.
```

Learnings, room for improvement, next steps

Key learnings

- Bob identified the seeded defect's root cause without diagnostic hints and reasoned carefully about migrations.
- Verify Bob's fixes by measuring them: two fixes looked right but failed until Bob saw the evidence.
- Bob-made skills, a command, a mode and a hook made later work faster and checkable.

Room for improvement

- No authentication or team workspaces yet.
- SQLite suits one worker; scaling out needs Postgres.
- No pagination or search in the board.

Next steps

- Accounts and workspaces, pagination and search, a Postgres option, overdue notifications.

Live demo: <https://kltamu.pythonanywhere.com> · Repository: <https://github.com/KnigguKniggu-droid/taskflow>